

Proceso de entrenamiento

Cómo se entrenó cada modelo del proyecto: datos, optimización, exportación int8 y variantes de entrenamiento.

Todos los modelos aprendidos excepto el TCN_MHA heredado se entrenaron con src/train.py siguiendo el procedimiento de abajo. Los documentos por familia dan los ajustes, la curva de aprendizaje y el resultado de cada ejecución.

1. Datos y ejemplos

- Datos: series RR en data/series/*.txt divididas por sujeto (142 de entrenamiento, 16 de validación; los sujetos con un solo registro van a validación). El entrenamiento usa solo los sujetos de entrenamiento; la validación vigila la parada temprana y se evalúa al final. Los 10 sujetos de test nunca se abrieron durante el entrenamiento.
- Ventanas: el motor en streaming recorre cada serie; cada latido con una ventana completa de 30 latidos da un ejemplo (características del latido n, objetivo dRR[n+1]). Se descartan las primeras 30 ventanas de cada serie (RLS en frío). 20 755 925 ventanas de entrenamiento y 1 421 225 de validación.
- Normalización: las estadísticas z (RR, dRR, cada característica, el objetivo) se ajustan solo con los sujetos de entrenamiento y se guardan en el config.json de la ejecución; una varianza cero da desviación 1.
- Objetivo: $dRR[n+1] = RR[n+1] - RR[n]$. Los modelos residuales (todos menos linear) aprenden la corrección normalizada z dRR - rls_pred; la predicción vuelve a sumar rls_pred.
- RAM: el tensor de secuencias (N, 30, 2) nunca se construye. Cada ventana se guarda como sus características, su objetivo y el índice de inicio en la serie RR concatenada; los lotes se forman al vuelo con tf.data, y una aserción comprueba que este camino es igual al usado en inferencia (utils.make_inputs).

2. Optimización (modelos Keras: mlp, gru, micro_tcn, tcn_lite)

ajuste	valor
optimizador	AdamW, decaimiento de pesos 1e-4
pérdida	Huber, delta 1 (en unidades z: cuadrática hasta aproximadamente una desviación estándar del objetivo y lineal más allá, así que los latidos atípicos pesan menos que con MSE)
lote	512 ventanas; 64 pasos por llamada de Keras (steps_per_execution) para eliminar la sobrecarga por paso que domina en modelos pequeños
época	desde la ronda 2: 2 000 000 de ventanas extraídas uniformemente al azar CON reemplazo (3906 pasos, alrededor del 10% de los datos, un subconjunto nuevo en cada época). Ejecuciones de la ronda 1 (linear, mlp, gbd, micro_tcn, tcn_lite): una pasada completa por todas las ventanas de entrenamiento
tasa de aprendizaje	1e-3 desde cero, 3e-4 para ajustes finos y arranques en caliente; ReduceLROnPlateau la divide por dos cuando la pérdida de validación deja de mejorar (paciencia max(2, paciencia / 3) épocas), mínimo 1e-5
parada	EarlyStopping sobre la pérdida de validación, paciencia de 10 épocas (6 en los ajustes finos), restaurando los pesos de la mejor época; como máximo 100 épocas (30 en los ajustes finos, 90 en gru)
monitor de validación	un subconjunto aleatorio fijo de 200 000 ventanas de validación en cada época (desde la ronda 2); las métricas finales usan todas las ventanas de validación
reproducibilidad	semilla 211 para Python, NumPy, Keras y las extracciones de ventanas; operaciones de TensorFlow deterministas; el md5 de los manifiestos de datos y las versiones de las bibliotecas se guardan en config.json
hardware	una GPU (TensorFlow 2.19.1), un trabajo cada vez mediante la cola jobs/; tiempo por paso de 3 ms (mlp) a 25 ms (gru)

3. Modelos no neuronales

- linear: regresión ridge (penalización 1e-3, término independiente sin penalizar) resuelta de forma cerrada con ecuaciones normales en float64 acumuladas en bloques de 1M ventanas; sin épocas, 0.6 s.
- gbd: HistGradientBoostingRegressor de scikit-learn en CPU, parada temprana sobre las ventanas de validación (ver 04_gbd).

4. Después del entrenamiento (cada ejecución)

- El modelo guardado se recarga desde disco y se evalúa en ms sobre todas las ventanas de validación, junto a rls_ar (y al padre en los ajustes finos); todo va a runs/<tag>/config.json y la curva por época a history.csv.
- Exportación int8 (Keras): cuantización post-entrenamiento totalmente entera con TFLite (pesos, activaciones, entrada y salida int8). Desde el 2026-10-08 la calibración es del mejor de K: se exportan 8 extracciones fijas (2000 y 5000 ventanas de entrenamiento x 4 semillas) más una referencia simple de 20 000 ventanas, y se conserva la extracción con la menor diferencia int8-frente-a-float en ventanas de ENTRENAMIENTO reservadas: 20 sujetos de

entrenamiento excluidos de todas las extracciones, 20 000 ventanas ordinarias + 20 000 de lazo cerrado rellenas por el propio modelo float. Una aserción exige que el mejor de K no sea peor que la referencia. La validación nunca participa en la elección.

- Historia de la calibración: las ejecuciones de la ronda 1 se exportaron con el primer train.py; desde el 2026-10-07 a las 18:14 un error calibraba con solo 500 ventanas (salida int8 saturada, diferencias del 3.0-3.7%), corregido esa misma tarde con 20 000 ventanas y train.py --reexport_int8; el mejor de K lo sustituyó el 2026-10-08 para los finalistas de las rondas 3/4. El documento de cada ejecución indica qué exportación tiene.

5. Variantes de entrenamiento usadas en rondas posteriores

- **Ajuste fino** (--init_from): continuar desde los pesos de una ejecución entrenada con lr 3e-4, como máximo 30 épocas, paciencia 6 (tcn_lite_ft).
- **Arranque en caliente con entradas extra** (--init_from + --features): las entradas nuevas reciben pesos cero y su propia normalización ajustada en entrenamiento, así que el entrenamiento parte de la función exacta del padre (ejecuciones_rsa_ws).
- **Datos de lazo cerrado, DAD** (--dad_filler, --dad_share 0.5): las series de entrenamiento se cortan en segmentos de 5000 latidos, se elimina el 10-80% de los latidos y los rellena el modelo padre como en el marcapasos, el motor recalcula las características sobre la serie rellena y el objetivo sigue siendo el latido siguiente VERDADERO (rr_true[n+1] - rr_filled[n]); la mitad de las ventanas de los lotes viene de estos segmentos (ejecuciones_dad). Un primer intento solo alcanzaba las primeras 429k ventanas de cada mitad (un error de rango de índices en tf.data) y se borró; ahora una comprobación detiene el entrenamiento si las extracciones no pueden alcanzar todas las ventanas.

Documentos de esta carpeta (cada uno en inglés _en y en español _es)

archivo	contenido
00_training_pipeline	este documento: el procedimiento común
01_baselines	persistence, ema_mean, rls_ar: sin entrenamiento; cómo se adapta rls_ar en línea
02_linear	linear
03_mlp	mlp, mlp_nobp, mlp_rsa
04_gbdtd	gbdt, gbdtd_all
05_gru	gru
06_micro_tcn	micro_tcn, micro_tcn_f8, micro_tcn_dad, micro_tcn_rsa, micro_tcn_rsa_ws
07_tcn_lite	tcn_lite, tcn_lite_ft (seleccionado), tcn_lite_dad, tcn_lite_rsa, tcn_lite_rsa_ws
08_tcn_mha	el TCN_MHA heredado (entrenado fuera de este proyecto)

Abreviaturas y siglas usadas en este documento

término	significado
AdamW	optimizador Adam con decaimiento de pesos desacoplado (algoritmo de entrenamiento de los modelos neuronales)
ar	nombre de conjunto de características: entradas autorregresivas (rr_dev y los 8 dRR más recientes) del modelo lineal
CPU	unidad central de procesamiento
DAD	data as demonstrator: entrenamiento con entradas que contienen los rellenos de lazo cerrado del propio modelo, con el objetivo verdadero (remedio contra el sesgo de exposición)
_dad	sufijo de nombre de ejecución: ajuste fino con datos DAD (rellenos en lazo cerrado)
dRR	cambio latido a latido del intervalo RR: $dRR[n+1] = RR[n+1] - RR[n]$ (ms); la magnitud que se predice
ema_mean	referencia que predice el siguiente RR como media móvil exponencial de la ventana
_f8	sufijo de nombre de ejecución: 8 filtros de convolución en lugar de 16
float64	números de coma flotante de 64 bits (doble precisión)
_ft	sufijo de nombre de ejecución: ajuste fino (entrenamiento continuado desde los pesos del modelo padre)
gbdt	árboles de decisión potenciados por gradiente (nombre de ejecución)
GPU	unidad de procesamiento gráfico
gru	unidad recurrente con compuertas (nombre de ejecución)
HRV	variabilidad de la frecuencia cardíaca
int8	entero de 8 bits: pesos y activaciones cuantizados a enteros de 8 bits, como se ejecuta en el microcontrolador; _int8 = la versión int8 de una ejecución
lr	tasa de aprendizaje (tamaño de paso del optimizador)
MHA	atención multicabeza (capa de tipo transformer, no soportada por la cadena de herramientas del MCU)
mlp	perceptrón multicapa (nombre de ejecución)
ms	milisegundos
MSE	error cuadrático medio
nobp	parte del nombre de ejecución: sin pasa banda (se quitan las características bp y bp_prev del pasa banda RSA fijo)
RAM	memoria de acceso aleatorio
RLS	mínimos cuadrados recursivos: filtro adaptativo que actualiza sus coeficientes en cada latido
rls_ar	referencia: pronóstico AR(8) adaptativo por paciente mediante RLS
rls_pred	característica: pronóstico RLS AR(8) del siguiente dRR (ms), recortado a +-300
RR	intervalo RR: tiempo entre dos latidos consecutivos (ondas R del ECG), en ms
_rsa	sufijo de nombre de ejecución: usa las características del seguidor de RSA
TCN	red convolucional temporal (convoluciones 1-D causales y dilatadas)
tcn	red convolucional temporal (parte del nombre de ejecución)
TFLite	TensorFlow Lite: entorno de ejecución y formato de archivo para modelos en dispositivos embebidos
_ws	sufijo de nombre de ejecución: arranque en caliente (inicializado con los pesos del padre, entradas nuevas a cero)
z	puntuación z: (valor - media) / desviación estándar, con estadísticas de los sujetos de entrenamiento